Exercício – pré condições em programas Python

O objetivo desse exercício é praticar dus boas práticas para programas Python. A primeira é a declaração de tipos para parâmetros e retornos de métodos. A segunda é o uso de asserts para a definição de pré-condições para os parâmetros desses métodos.

Abaixo é apresentada uma especificação de uma classe que implementa uma tabela hash usando uma lista para armazenar pares de (chave,valor). Em seguida, é proposto o exercício a ser realizado.

Especificação funcional

Classe TabelaHashSondagemLinear

Representa uma tabela de associação entre chaves e valores. Cada chave identifica, no máximo, um valor.

A tabela possui capacidade fixa e resolve colisões por sondagem linear: quando a posição inicialmente calculada está ocupada por outra chave, as posições seguintes são examinadas sequencialmente, retornando ao início da tabela quando necessário.

A remoção é lógica. A posição removida recebe uma marca especial para não interromper a procura por chaves que possam ter sido armazenadas posteriormente na mesma sequência de sondagem.

As chaves podem ser:

- textos de até 16 caracteres, formados exclusivamente por letras do alfabeto inglês, algarismos ou sublinhado;
- números inteiros não negativos;
- números reais não negativos.

Valores lógicos não são aceitos como chaves. Os valores associados às chaves podem ser de qualquer tipo.

A tabela não aumenta automaticamente de tamanho.

Criação da tabela

Cria uma tabela vazia com a capacidade informada.

- Argumento: capacidade, do tipo inteiro positivo.
- Valor padrão: 11 posições.
- Retorno: nenhum.
- Estado inicial:
    - todas as posições estão vazias;
    - a quantidade de elementos armazenados é zero.

Função de hash

Calcula a posição inicial associada a uma chave.

- Argumento: uma chave válida.

- Retorno: um número inteiro entre zero e a capacidade da tabela menos um.

Para uma chave inteira, utiliza o resto da divisão da chave pela capacidade da tabela.

Para uma chave textual ou real, converte a chave em texto, soma os códigos numéricos de seus caracteres e utiliza o resto da divisão dessa soma pela capacidade.

Procura de posição

Procura internamente uma chave ou uma posição apropriada para inseri-la.

- Argumentos:
    - uma chave válida;
    - uma indicação lógica que informa se a procura se destina a uma inserção. O valor padrão dessa indicação é falso.
- Retorno:
    - o índice de uma posição da tabela; ou
    - ausência de resultado, quando a chave não for encontrada ou não houver posição disponível, conforme o tipo da procura.

Quando a procura não se destina à inserção:

- retorna a posição da chave, caso ela exista;
- informa ausência de resultado, caso a chave não exista.

Quando a procura se destina à inserção:

- retorna a posição atual da chave, caso ela já exista;
- retorna a primeira posição removida encontrada durante a sondagem, caso seja possível reutilizá-la;
- caso não tenha sido encontrada uma posição removida, retorna a primeira posição que nunca foi ocupada;
- informa ausência de resultado se todas as posições estiverem ocupadas e não houver posição removida.

Inserção

Insere uma associação entre uma chave e um valor.

- Argumentos:
    - uma chave válida;
    - um valor de qualquer tipo.
- Retorno: nenhum.

Se a chave ainda não estiver presente, a associação é armazenada e a quantidade de elementos aumenta em uma unidade.

Se a chave já existir, seu valor é substituído, sem alterar a quantidade de elementos.

Se não houver uma posição disponível, é sinalizado um erro de tabela cheia.

Busca

Obtém o valor associado a uma chave.

- Argumento: uma chave válida.
- Retorno: o valor associado, que pode ser de qualquer tipo.

Se a chave não estiver presente, é sinalizado um erro de chave inexistente.

Remoção

Remove logicamente uma chave e seu valor associado.

- Argumento: uma chave válida.
- Retorno: nenhum.

A posição não volta ao estado de nunca utilizada; ela passa a ser identificada como removida. A quantidade de elementos diminui em uma unidade.

Se a chave não estiver presente, é sinalizado um erro de chave inexistente.

Verificação de pertencimento

Verifica se determinada chave está armazenada.

- Argumento: uma chave válida.
- Retorno: um valor lógico.

O resultado é verdadeiro quando a chave está presente e falso caso contrário.

Verificação de pertencimento de valor

Verifica se determinado valor está armazenado na tabela.

- Argumento: um valor não modificável, de qualquer tipo.
- Retorno: um valor lógico.

O resultado é verdadeiro quando for encontrado um valor igual ao procurado e falso caso contrário. Posições vazias ou removidas não são consideradas.

Obtenção das chaves

Obtém todas as chaves atualmente armazenadas.

- Argumentos: nenhum.
- Retorno: uma lista de chaves.

As chaves aparecem na ordem das posições físicas da tabela, e não necessariamente na ordem em que foram inseridas. Posições vazias ou removidas não são incluídas.

Obtenção dos valores

Obtém todos os valores atualmente armazenados.

- Argumentos: nenhum.
- Retorno: uma lista de valores de quaisquer tipos.

Os valores aparecem na ordem das posições físicas da tabela. Posições vazias ou removidas não são incluídas.

Obtenção dos itens

Obtém todas as associações atualmente armazenadas.

- Argumentos: nenhum.
- Retorno: uma lista de pares, sendo cada par formado por uma chave e seu valor.

Os pares aparecem na ordem das posições físicas da tabela. Posições vazias ou removidas não são incluídas.

Quantidade de elementos

Obtém a quantidade de associações atualmente armazenadas.

- Argumentos: nenhum.
- Retorno: um número inteiro não negativo.

Posições marcadas como removidas não são contabilizadas.

Representação textual

Produz uma representação textual completa da tabela.

- Argumentos: nenhum.
- Retorno: um texto.

O texto contém uma linha para cada posição, em ordem crescente de índice. Cada linha apresenta:

- o índice e a indicação de que a posição nunca foi ocupada;
- o índice e a indicação de que a posição foi removida; ou
- o índice, a chave e o valor armazenado.

Exercício

Cabeçalhos e verificação dinâmica de tipos

Implemente apenas a interface da classe `TabelaHashSondagemLinear`. Para cada método da especificação:

1. Escreva sua declaração com `def`.
2. Acrescente anotações para todos os parâmetros.
3. Declare o tipo de retorno.
4. Use `assert` no início do método para verificar dinamicamente os tipos dos argumentos recebidos.
5. Não implemente o comportamento da tabela hash. Depois das verificações, encerre o método com `pass`.

Devem ser criados os cabeçalhos dos seguintes métodos:

```
__init__
funcao_hash
_procurar_posicao
inserir
buscar
remover
contem
contem_valor
chaves
valores
itens
__len__
__str__
```

Tipos que devem ser utilizados

- Uma chave pode ser um texto, um número inteiro ou um número real.
- Valores lógicos devem ser rejeitados como chaves, apesar da relação existente entre valores lógicos e inteiros na linguagem.
- Um valor associado a uma chave pode ser de qualquer tipo.
- A capacidade deve ser um inteiro positivo.
- O indicador de procura para inserção deve ser lógico.
- Uma posição deve ser representada por um inteiro ou pela ausência de valor.
- Os métodos de consulta coletiva devem retornar listas.
- Cada item da tabela deve ser representado por um par contendo uma chave e um valor.

Verificações esperadas

Método	Verificações com `assert`
`__init__`	A capacidade é inteira, não é lógica e é positiva
`funcao_hash`	A chave é textual, inteira ou real e não é lógica
`_procurar_posicao`	A chave possui tipo permitido e o indicador de inserção é lógico
`inserir`	A chave possui tipo permitido; o valor não exige verificação por aceitar qualquer tipo
`buscar`	A chave possui tipo permitido
`remover`	A chave possui tipo permitido
`contem`	A chave possui tipo permitido
`contem_valor`	O valor procurado é `Hashable`, sendo tratado como um valor não modificável
`chaves`	Não recebe argumentos que exijam verificação
`valores`	Não recebe argumentos que exijam verificação
`itens`	Não recebe argumentos que exijam verificação
`__len__`	Não recebe argumentos que exijam verificação
`__str__`	Não recebe argumentos que exijam verificação